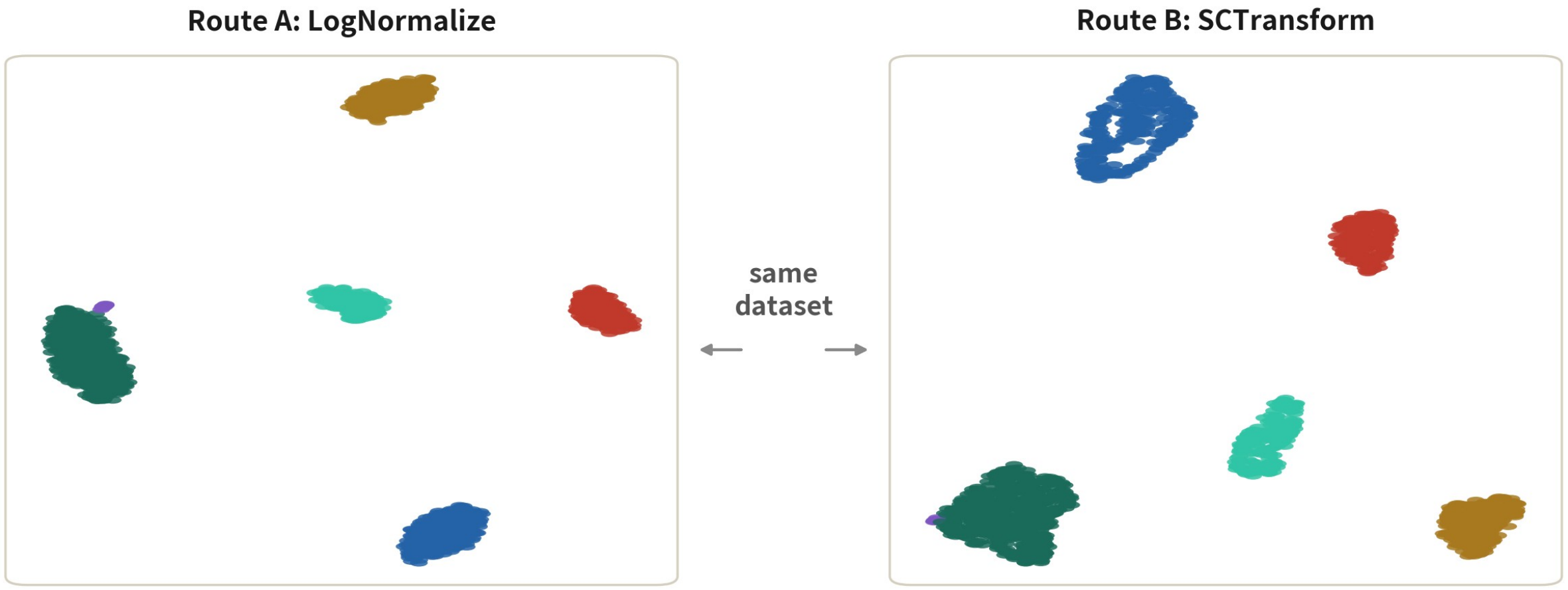
B6 Series B

Normalization, HVG, Scaling: two routes, tested head-to-head

Run LogNormalize and SCTransform side by side, compare downstream, and know when to use which.

About 38 min | Prerequisites: B5 | Data: PBMC 3k (post-QC)

Same data, two normalization routes



simulated data

Same cells, same seed — different cluster shapes. Which one is "right"?

Left: LogNormalize. Right: SCTransform. The shapes differ — which one is 'right'?

WHERE THIS EPISODE STARTS

'Which is right?' — wrong question.

Both are right; they answer different models.

The skill is not picking a camp — it is knowing what each route does, how downstream differs, and which fits your data.

What you will learn

1. Explain what each of NormalizeData, FindVariableFeatures and ScaleData solves

2. Run SCTransform and state which three steps it replaces and how it differs

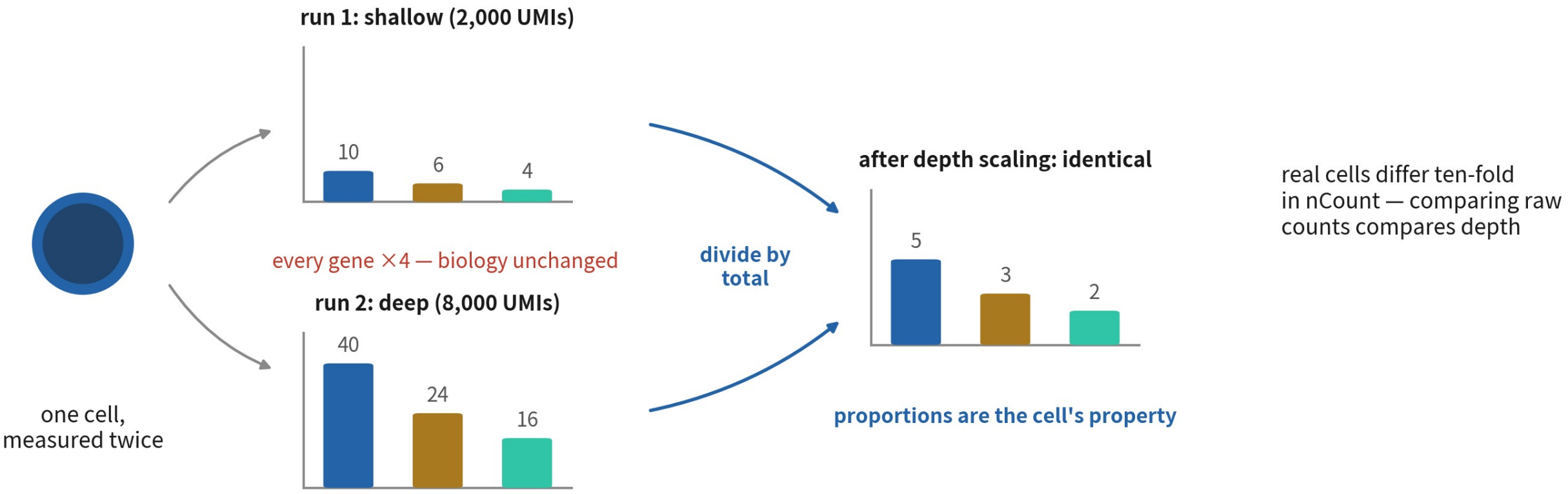
3. Compare HVG overlap and UMAPs across both routes on the same data, then choose with reasons

01

Three problems, three steps

Before any code: know what mess each step cleans up

Problem 1: depth differs — raw counts don't compare



The same cell measured twice: counts differ four-fold, biology unchanged. Proportions are the cell's property

Three problems, three steps

problem 1

depths differ —
raw counts not comparable

↓

`NormalizeData`

divide by total ×10,000
then log1p → data layer

→

problem 2

20k genes,
most carry only noise

↓

`FindVariableFeatures`

rank by vst, keep
2,000 HVGs

→

problem 3

magnitudes differ —
loud genes dominate

↓

`ScaleData`

z-score per gene,
clip at ±10 → scale.data

each step eats the last: counts → data → HVG list → scale.data → PCA (B7)

Each step eats the previous step's output — no reordering, no skipping

CORE IDEA

These steps produce no biological conclusions — yet every downstream conclusion is built on them.

PCA eats scale.data, clustering eats PCA, annotation eats clustering. Tilt the foundation one degree and the roof tilts ten.

02

NormalizeData: divide depth out

CP10K + log1p, two moves in one line

Pick up from B5: load the post-QC object

```
library(Seurat)
set.seed(1234)                # 全系列固定

pbmc <- readRDS("output/pbmc_b05_qc.rds")
pbmc
```

```
An object of class Seurat
13714 features across 2638 samples within 1 assay
Active assay: RNA (13714 features, 0 variable features)
 1 layer present: counts
```

2,638 clean cells; only the counts layer — exactly this episode's raw material

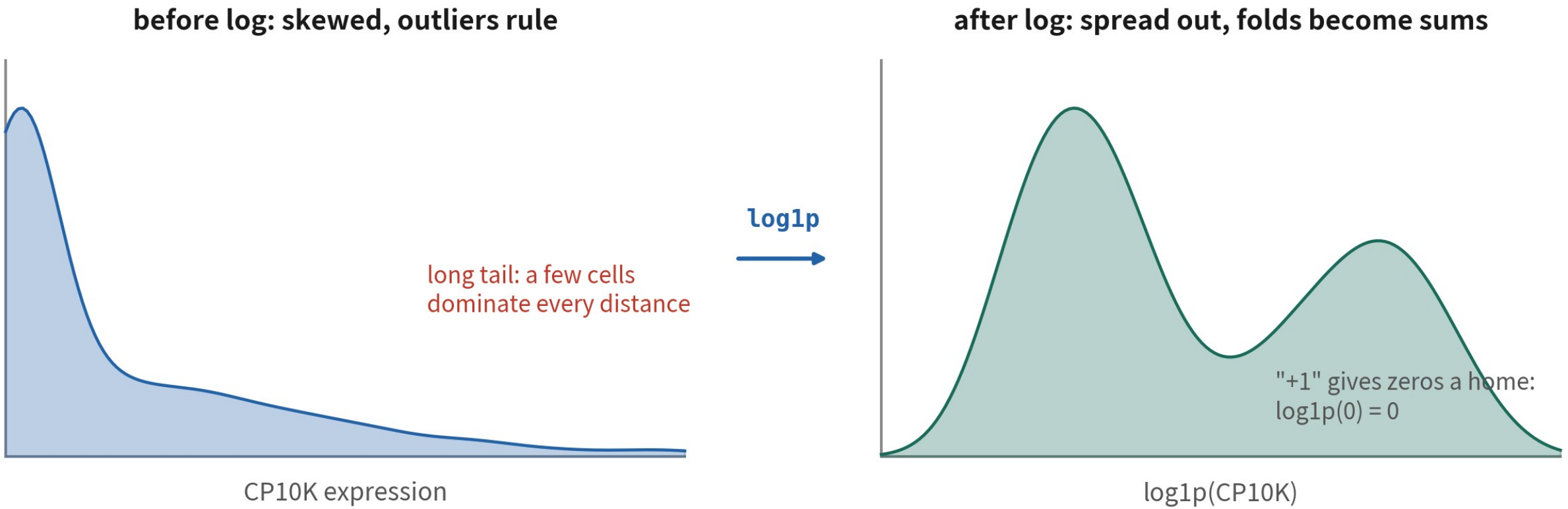
NormalizeData: one line, two moves

```
pbmc <- NormalizeData(pbmc,
      normalization.method = "LogNormalize",
      scale.factor = 10000)
```

```
Normalizing layer: counts
Performing log-normalization
0%   10   20   30   40   50   60   70   80   90   100%
[----|----|----|----|----|----|----|----|----|----|
```

Both arguments are defaults — spelled out so you know they exist and can be changed

Why log after dividing by total



simulated data

Log turns folds into sums and tames outliers; the +1 gives zeros a home

LayerData: before and after

```
LayerData(pbmc, layer= "counts")["LYZ", 1:3] #  原始
LayerData(pbmc, layer= "data")["LYZ", 1:3]  # normalize 後
```

```
AAACATACAACCAC-1 AAACATTGAGCTAC-1 AAACATTGATCAGC-1
              21                1               42
AAACATACAACCAC-1 AAACATTGAGCTAC-1 AAACATTGATCAGC-1
        4.475061         1.112256         4.901619
```

Counts stay untouched — B12's DE still needs them. data is a new layer

What this section did

counts intact

raw counts never change; DE and checks depend on them

data is born

log1p(CP10K): expression comparable across cells

who eats it downstream

FeaturePlot, VlnPlot and FindMarkers all read data by default

03

FindVariableFeatures: keep the genes that talk

Of 13k genes, most carry only noise

Rank by vst, keep the top 2,000

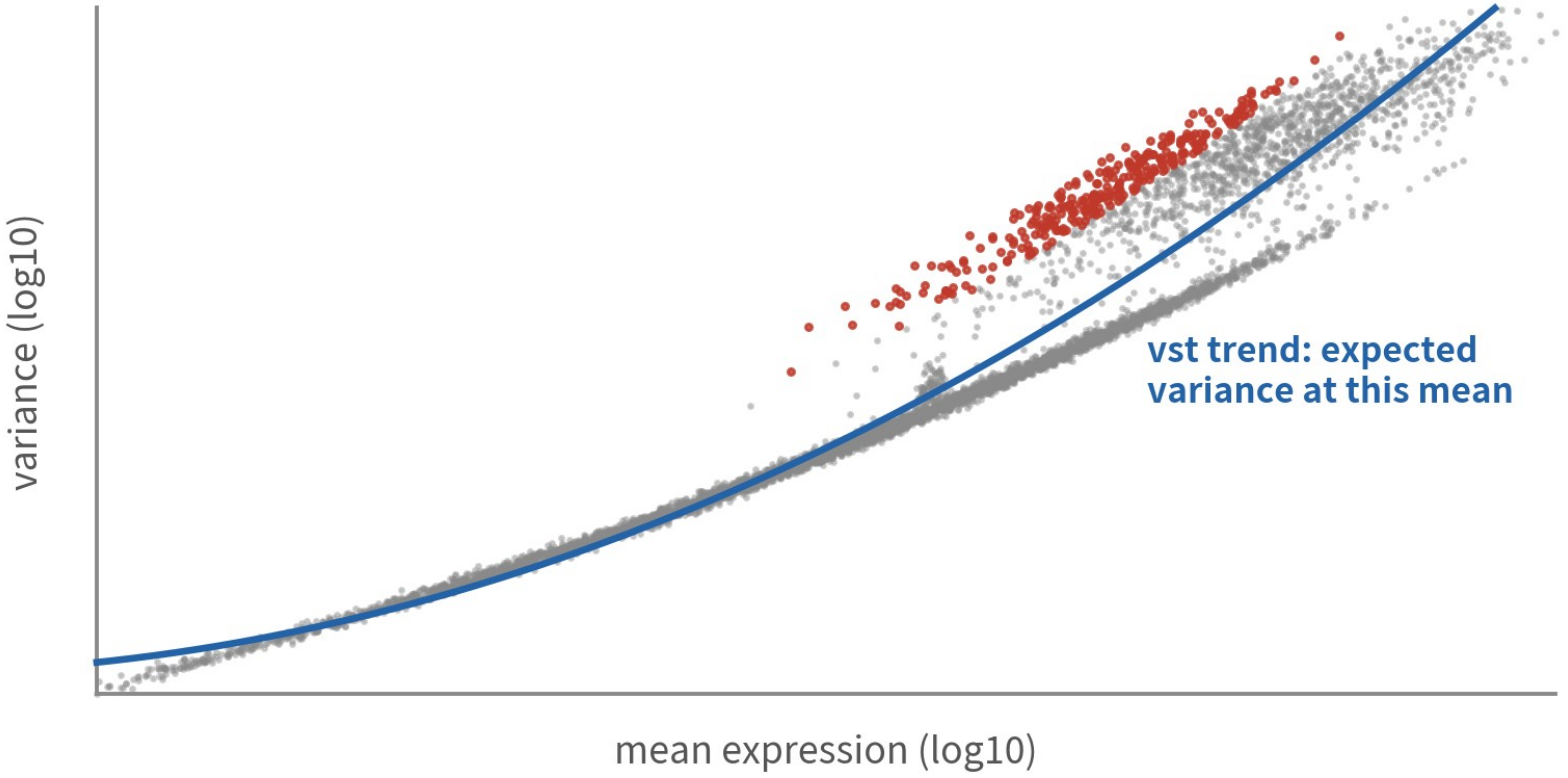
```
pbmc <- FindVariableFeatures(pbmc,
      selection.method = "vst", nfeatures = 2000)

top10 <- head(VariableFeatures(pbmc), 10)
top10
```

```
[1] "PPBP"  "LYZ"   "S100A9" "IGLL5" "GNLY"
[6] "FTL"   "PF4"   "FTH1"   "GNG11" "S100A8"
```

PPBP marks platelets, LYZ marks monocytes — the list itself is biology

What vst does: the mean-variance trend



grey: on the curve — variance explained by noise; skip

red: above the curve — more variable than expected → HVG

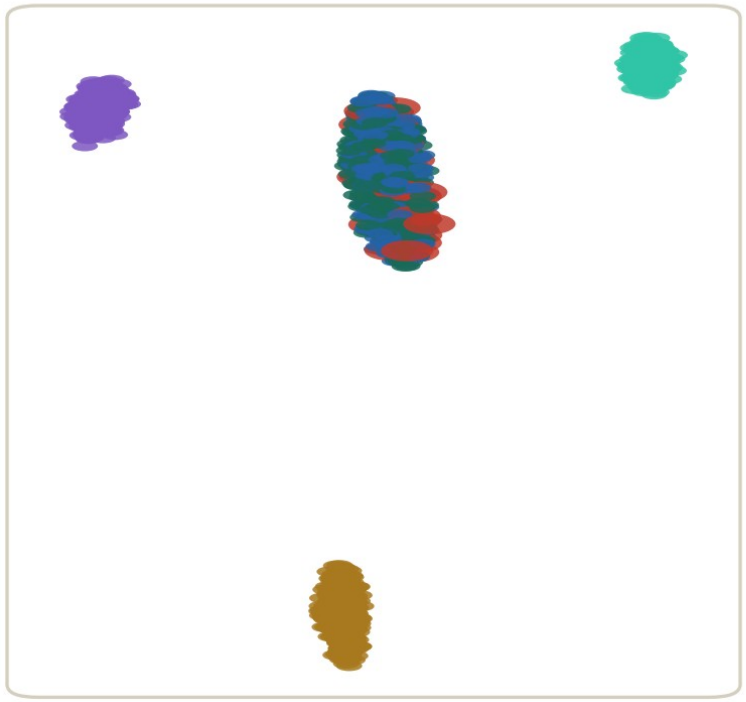
vst ranks by distance to the curve — decoupled from mean

simulated data

HVGs are genes more variable than their mean predicts — loud is not the same as informative

Where the 2,000 comes from

HVG = 200



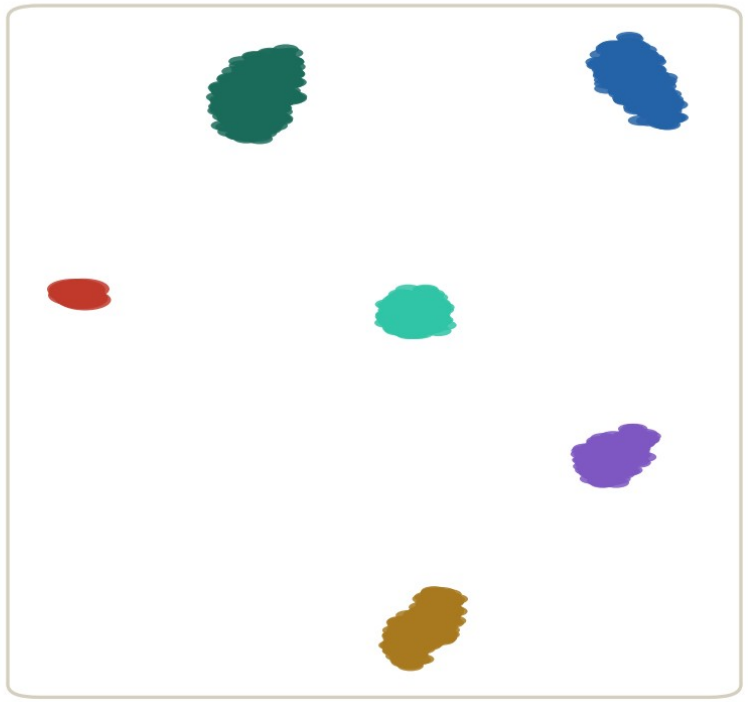
rare subtype (red) swallowed

red: the 2% rare subtype

HVG = 2000

rare subtype stands alone

HVG = 5000



nearly identical to 2000 — just slower

simulated data

Too few loses rare signal; too many is just slower. 2,000 is an empirically generous default

WHEN TO CHANGE NFEATURES

Start at the default 2,000; raise it when hunting rare populations — and write down why.

Raising it (3,000–5,000) only costs speed; lowering it can cost an entire population.

04

ScaleData: level the magnitudes

z-score + clip — paving the road for PCA

ScaleData: defaults to the HVGs

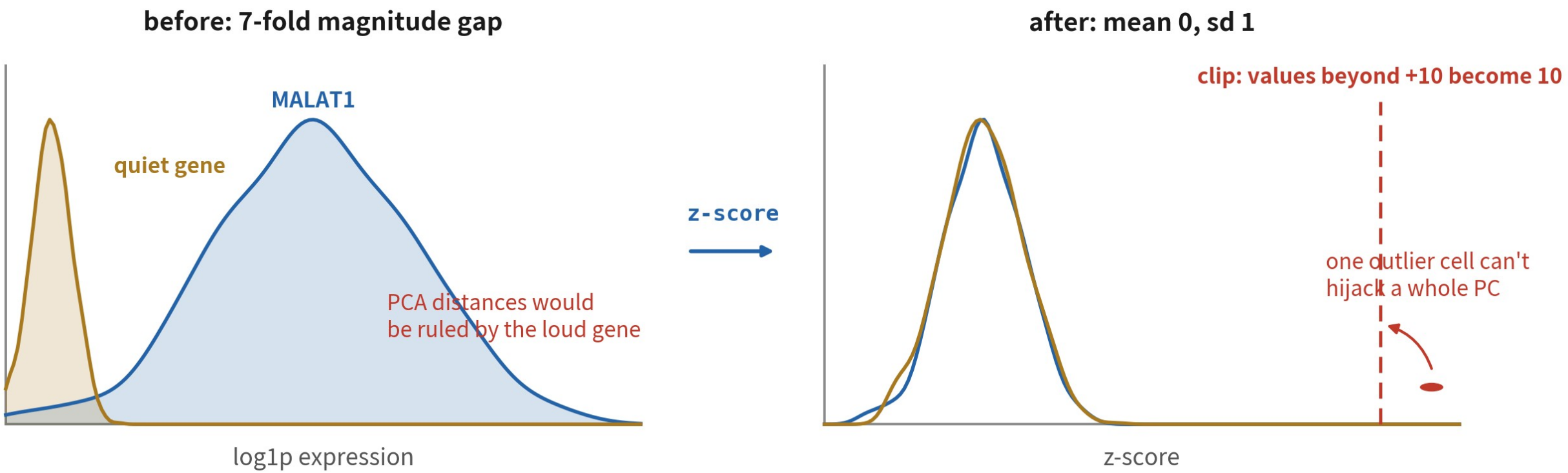
```
pbmc <- ScaleData(pbmc) # 預設 features = HVG 那 2000 個

LayerData(pbmc, layer = "scale.data")["LYZ", 1:3]
```

```
AAACATACAACCAC-1 AAACATTGAGCTAC-1 AAACATTGATCAGC-1
       0.9438414       -1.5018971        1.2110247
```

To scale all genes (for heatmaps): features = rownames(pbmc)

What z-score and clip each do



Z-scores give every gene an equal voice; the ±10 clip stops one outlier cell hijacking a PC

The vars.to.regress debate

Regressing out percent.mt or cell cycle — yes or no?

skip (our route)

don't regress by default

- QC already removed high-mt cells; residual effect is small
- transparent: nothing extra was subtracted
- fast, and easier to interpret

use with reasons

only after confirming the nuisance

- regress after clustering shows a cycle cluster or an mt gradient
- every regressed variable goes in the methods section
- regressed signal never comes back — be sure it is noise

All three layers present

counts

raw UMI counts: home base for DE (B12) and every check

data

log1p(CP10K): plotting and marker tests read here

scale.data

z-scored HVGs: consumed only by PCA (B7)

05

SCTransform: three steps in one

The other route — a one-liner

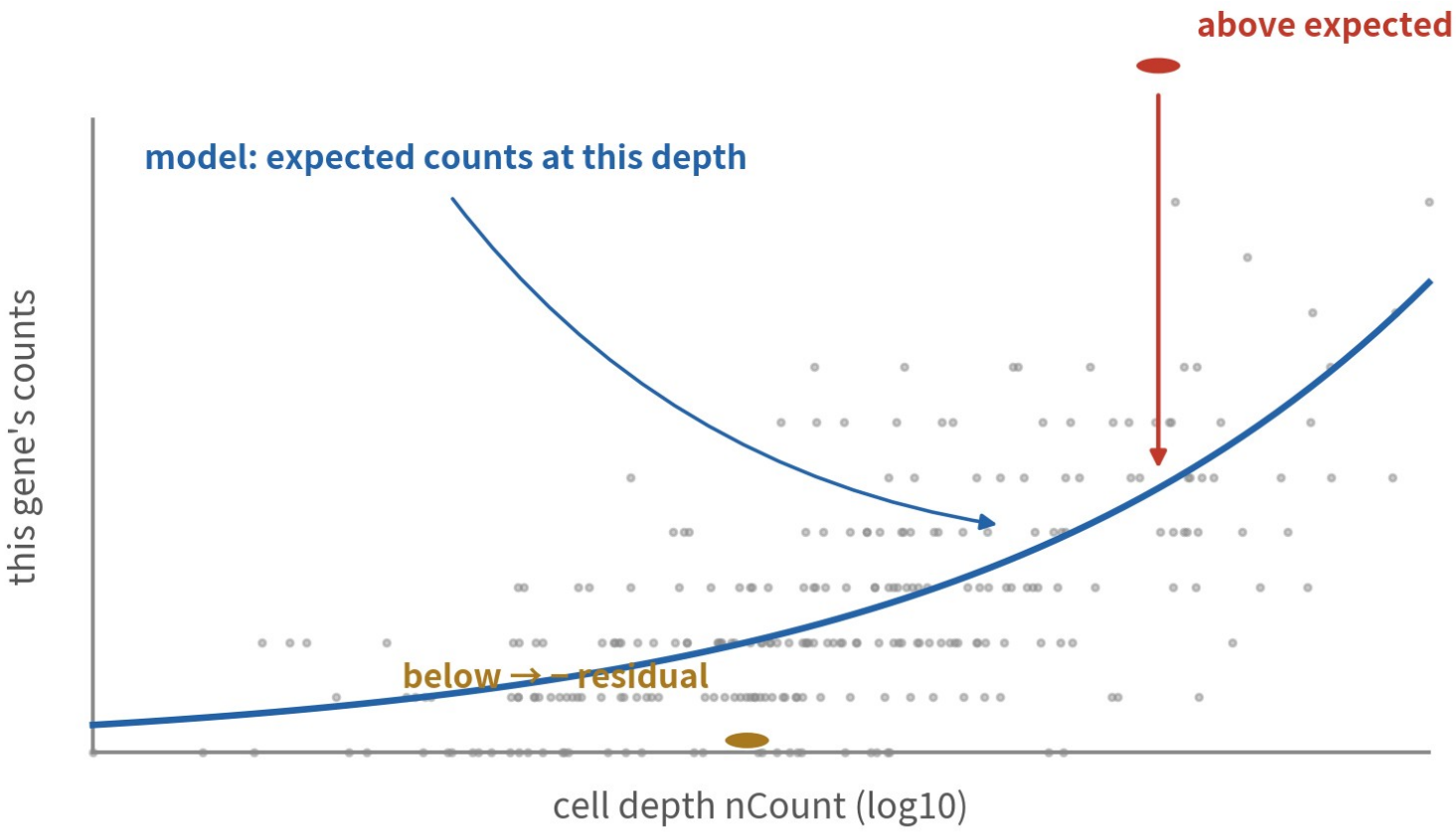
SCTransform: one line replaces three steps

```
# install.packages("sctransform") # 第一次需要
pbmc.sct <- SCTransform(pbmc, vst.flavor = "v2",
                        verbose = FALSE)
pbmc.sct
```

```
An object of class Seurat
26363 features across 2638 samples within 2 assays
Active assay: SCT (12649 features, 3000 variable features)
 3 layers present: counts, data, scale.data
 1 other assay present: RNA
```

A new SCT assay appears with its own three layers and 3,000 HVGs; the RNA assay is untouched

What residuals are, in plain words



- ✓ one depth-to-expectation curve per gene
- ✓ v2: parameters shared across genes (regularization) — quiet genes stay stable
- ✓ residual = (observed − expected) / expected sd
- → one move removes depth and stabilises variance = three steps in one

simulated data

Each gene gets a depth-to-expectation curve; the residual is how many sd you sit above or below it (math in C2)

Inside the SCT object: data is already logged

```
DefaultAssay(pbmc.sct)   # 後續操作都在 SCT assay
head(VariableFeatures(pbmc.sct), 3)

LayerData(pbmc.sct, layer = "data")["LYZ", 1:3]
```

```
[1] "SCT"
[1] "GNLY"  "LYZ"   "S100A9"
AAACATACAACCAC-1 AAACATTGAGCTAC-1 AAACATTGATCAGC-1
        3.135494         0.693147         3.401197
```

data = log1p(corrected counts); scale.data = residuals. Remember this — the traps section collects on it

THIS SECTION IN ONE LINE

SCT = normalization + HVG + scaling in one pass, living in its own assay.

The cost: one more black box, a bigger object, and DE must switch back to the RNA assay (see B12).

06

The two routes, head to head

Same data, three checkpoints

Checkpoint 1: how much do the HVG lists overlap?

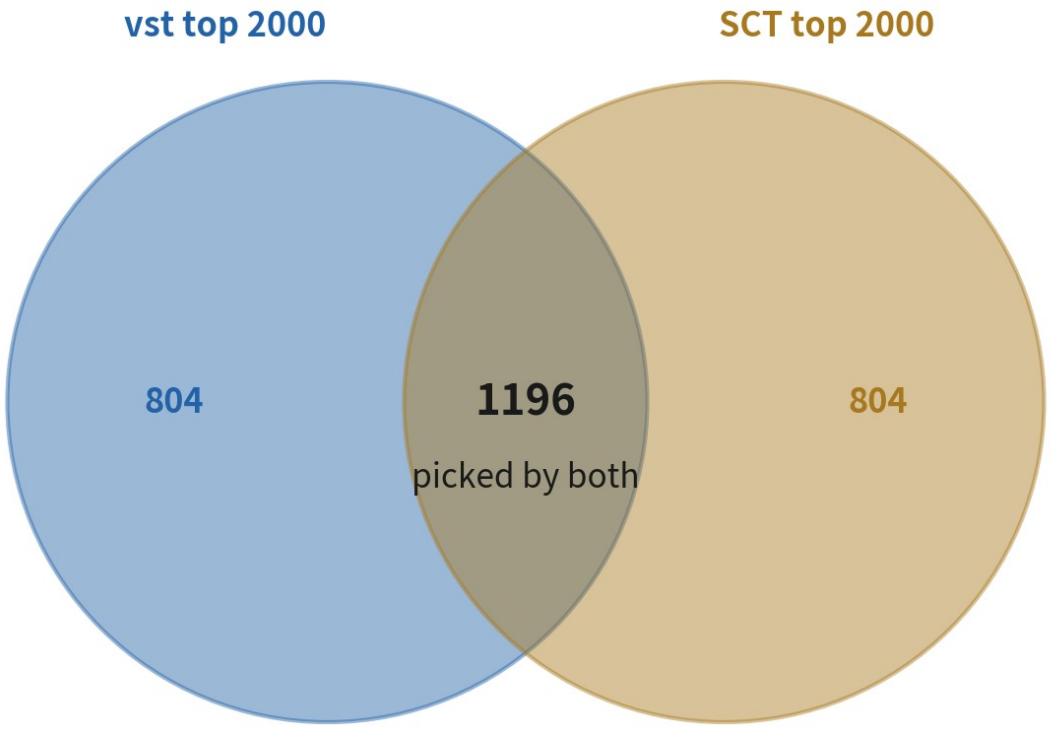
```
hvg.log <- VariableFeatures(pbmc)        # 2000 個
hvg.sct <- head(VariableFeatures(pbmc.sct), 2000)

length(intersect(hvg.log, hvg.sct))
```

```
[1] 1245
```

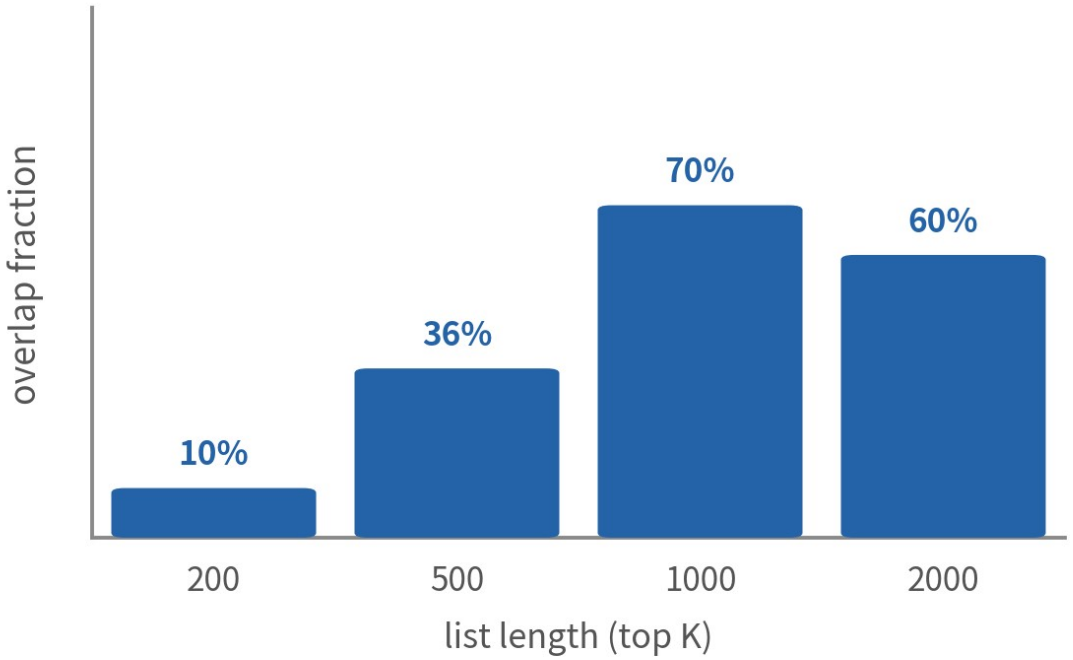
Roughly 60% overlap: solid consensus at the core, disagreement at the margins

How to read the overlap



60% overlap: strong consensus, not identical

the very top disagrees a lot; consensus builds as lists grow



simulated data

Strong genes are caught by both; disagreement sits in ranking and margins — the normal footprint of different models

Checkpoint 2: UMAPs side by side

```
pbmc   <- RunPCA(pbmc)   |> RunUMAP(dims = 1:10)
pbmc.sct<- RunPCA(pbmc.sct) |> RunUMAP(dims = 1:10)

library(patchwork)
p1 <- DimPlot(pbmc)   + ggtitle("LogNormalize")
p2 <- DimPlot(pbmc.sct)+ ggtitle("SCTransform")
p1 + p2
```

PCA parameters wait for B7 — here we just run defaults to get both pictures

Checkpoint 3: is downstream stable?

What the head-to-head on PBMC 3k actually shows

agrees

major structure is identical

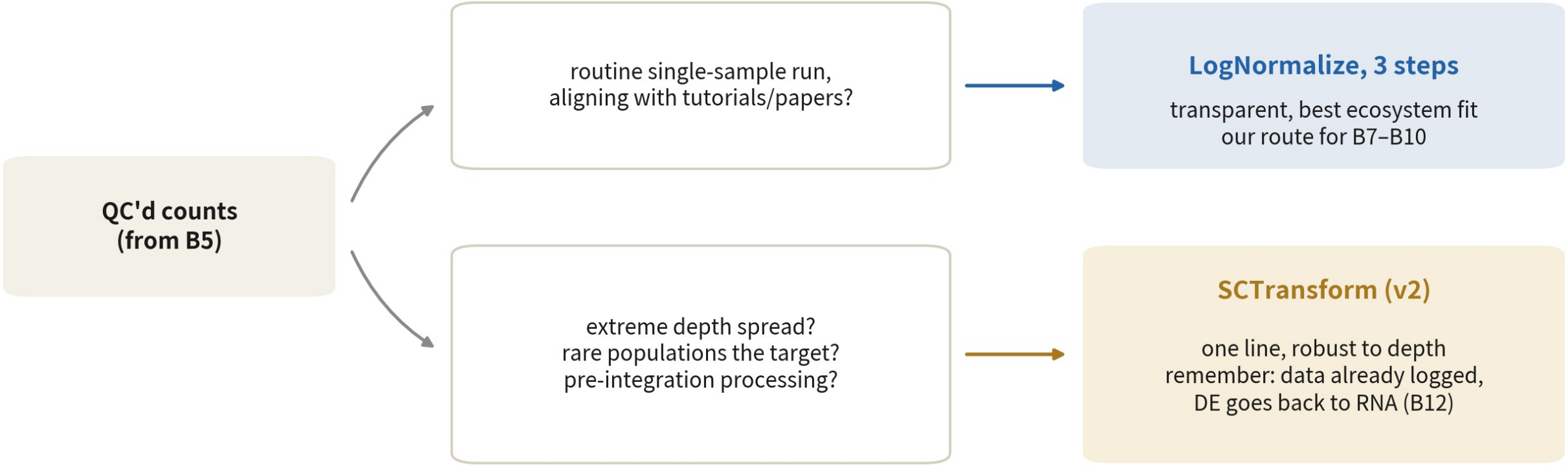
- T, B, NK and monocyte blocks all present
- canonical markers (CD3E, MS4A1, LYZ) clear in both
- cluster memberships overlap heavily

differs

shapes, spacing, details

- cluster shapes and layout differ — neither is wrong
- SCT places low-depth cells more stably
- borderline subgroups split at slightly different grain

How to choose: one decision map



both are legal; switching routes = rerun everything downstream, and write down why

Start routine work with LogNormalize; consider SCT for extreme depth spread, rare-cell hunts, or pre-integration

OUR CHOICE FOR THIS SERIES

B7–B10 run on LogNormalize — not because it is 'righter', but because it is transparent, teachable, and best supported.

You can always switch — just rerun downstream. The danger is not the wrong choice; it is switching and rerunning only half.

07

Where people get hurt

Three traps — each demonstrated for real

Trap 1: PCA straight on raw counts

What it is

Feeding raw counts into ScaleData + RunPCA, skipping NormalizeData.

Why it happens

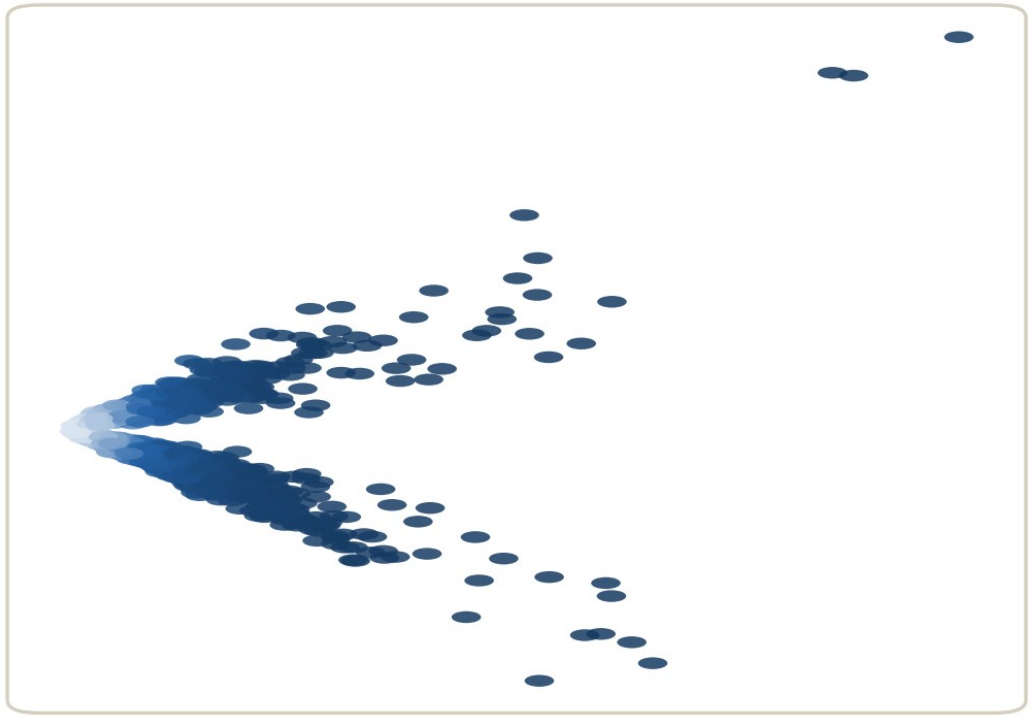
Everything runs, nothing errors, plots appear — it looks fine.

What it costs you

PC1 becomes the depth axis: downstream clusters separate sequencing depth, not cell types.

Live demo: PC1 versus nCount

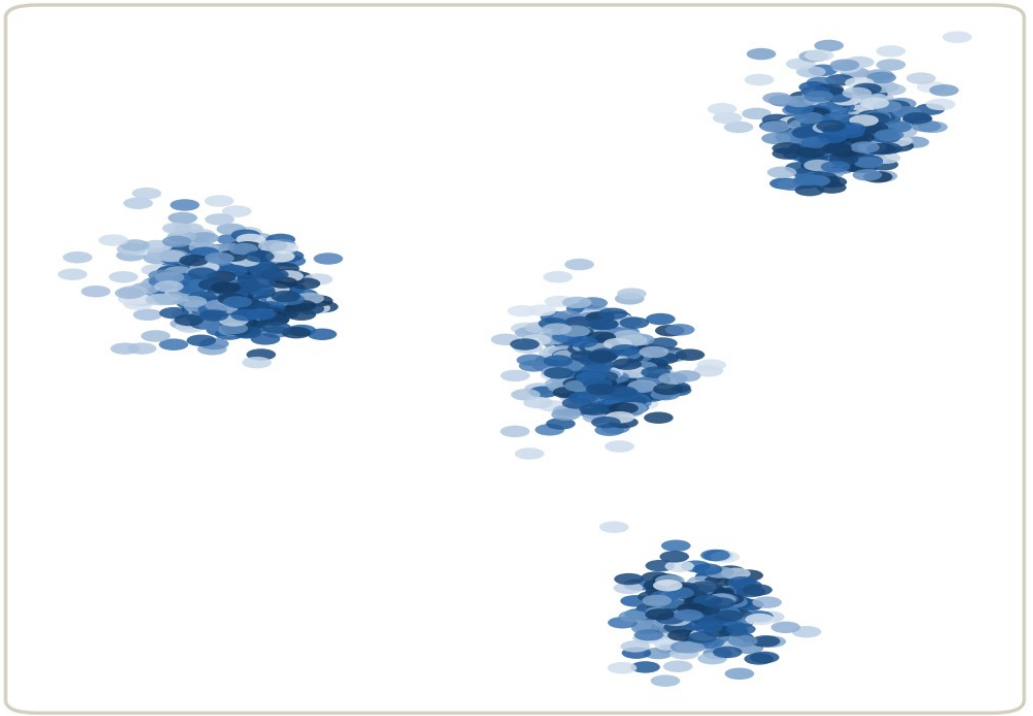
skip normalization: PC1 is depth



colour = depth (light → dark)

`cor(PC1, nCount) = 1.00`

after the three steps: depth exits



colour = depth (light → dark)

`cor(PC1, nCount) = 0.00`

simulated data

One-line check: cor(pc1, pbmc$nCount_RNA) — near 1 means you are hit

Trap 2: keeping only 200 HVGs

What it is

Cutting nfeatures to 200 to 'focus on what matters'.

Why it happens

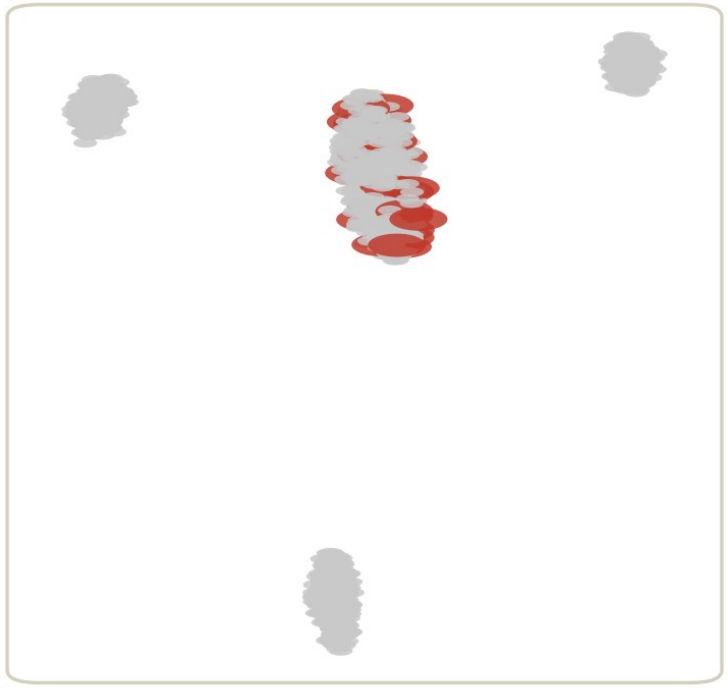
The top 200 are dominated by big-population markers — it feels sufficient.

What it costs you

Rare-subtype markers never crack the top 200; those cells lose their coordinates and dissolve into the parent cluster.

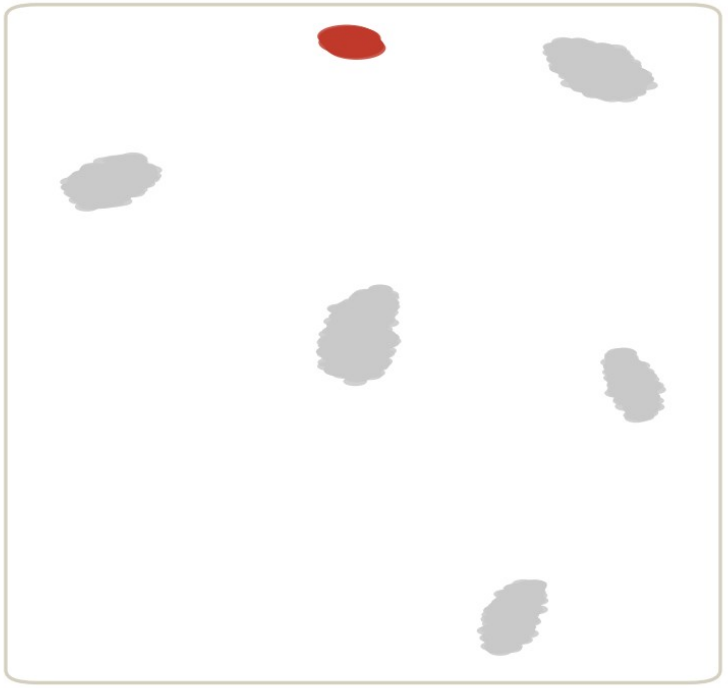
Live demo: the rare subtype vanishes

HVG = 200

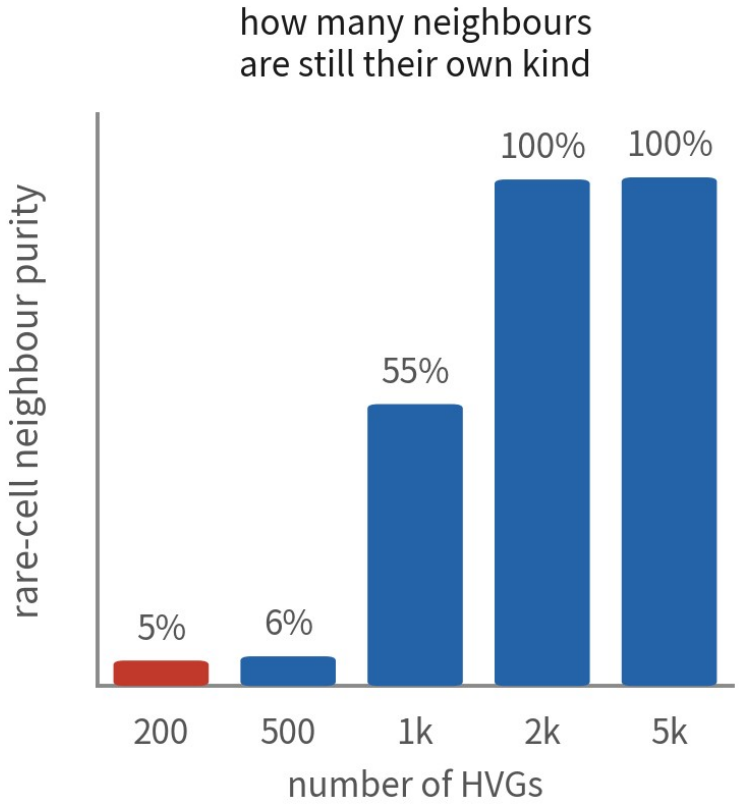


red cells dissolve into the crowd

HVG = 2000



red cells form their own island



simulated data

At 200 HVGs neighbour purity drops to 5% — the population effectively ceases to exist; at 2,000 it returns intact

Trap 3: manual log after SCT

What it is

Running NormalizeData or log1p() on SCT's data layer.

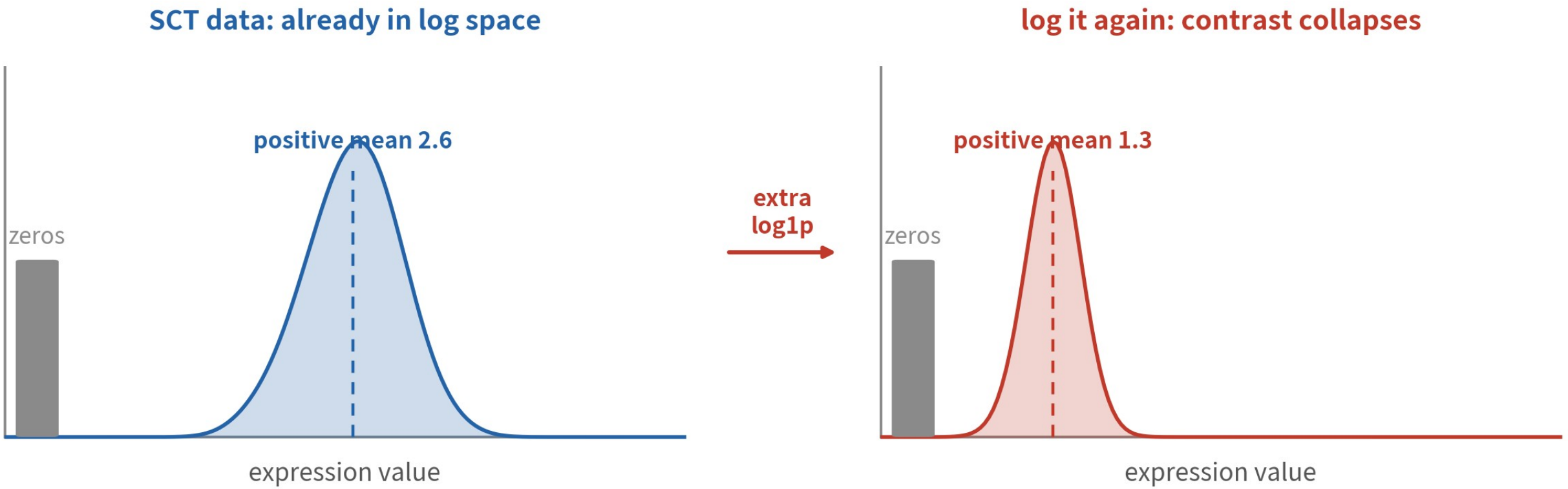
Why it happens

Muscle memory: 'got expression values? log them first.'

What it costs you

Double-logging crushes contrast: the gap between positive and negative cells collapses and FeaturePlots wash out.

Live demo: the double-transformed distribution



simulated data

positive-negative gap: 2.6 → 1.3; FeaturePlots wash out

Rule of thumb: SCT's data ships pre-logged — so does LogNormalize's. Never log twice

TRAPS, SUMMED UP

None of these traps throws an error.

Your checks catch them — the code will not.

The lifesaver: cor(Embeddings(pbmc,"pca")[,1], pbmc$nCount_RNA). Run it after every PCA.

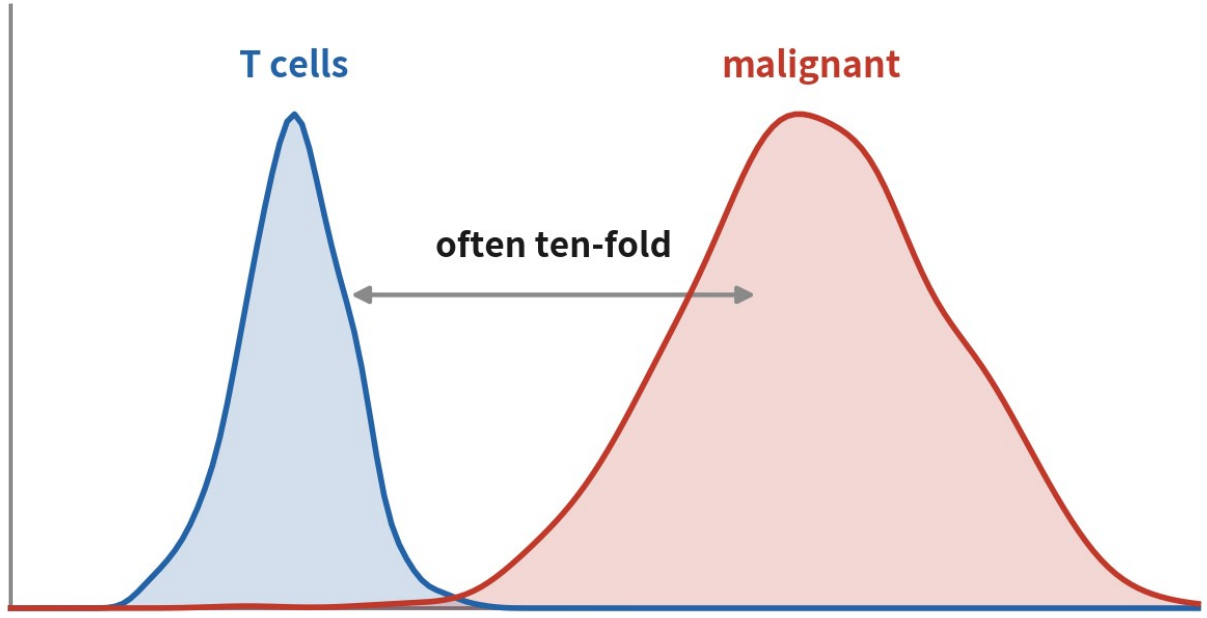
5+

Strategy Room: Complex Samples

The standard route is done — now imagine what you hold is not PBMC, but a tumour

Malignant library size: an assumption under strain

nCount inside one tumour: two populations, two magnitudes



illustrative distributions

the assumption

LogNormalize and SCT both assume:
total RNA = technical, not biology

tumour reality

malignant cells are big, often polyploid,
transcription globally up — 'depth' holds biology

the decision

cor(PC1, nCount) 0.3–0.5 is common in tumours:
judge by PC1 loadings; only > 0.5 sends you back

Normalization assumes total RNA is not biology — malignant cells break exactly that. Judge by loadings, not by the coefficient

Regress cell-cycle scores? A decision tree

Run CellCycleScoring during preprocessing — scores only, no regression yet

after clustering, check two things

1. DimPlot coloured by Phase: is S / G2M confined to one small cluster?
2. PC1–PC5 loadings: any cycle genes like MKI67 or TOP2A?

both are clean

don't regress — label that cluster 'cycling X'
in tumours proliferation is often the signal:
grade, aggressiveness, treatment response

every type split in half

cycle hijacked the clustering — regress S and
G2M scores, or regress only 'S − G2M'
to keep cycling-vs-not information

regressed signal never comes back — think about the question before vars.to.regress, and record it

In tumours proliferation is signal, not noise: think about the research question before touching vars.to.regress

SCT on tumour data: the trade-off

Big depth spread argues for SCT — and the red lines multiply

where it gains

tumours live in the big-depth-spread box

- malignant vs T cells often differ ten-fold in RNA: depth comes out cleaner
- HVG selection is steadier; v2 is built for exactly this spread
- low-depth cells and weak signals usually survive better

red lines

much of the tumour downstream eats raw counts only

- DE (B12), inferCNV (B16), pseudobulk: all go back to the RNA assay's counts
- run PrepSCTFindMarkers before FindMarkers
- multi-sample integration: run SCT per sample (B11)

HVGs hijacked by the patient effect: per-sample union

the symptom

pool tumour patients and pick HVGs — the top of the list is patient-to-patient difference

why

malignant cells cluster by patient: every patient's CNVs and genome differ

the strategy

pick HVGs per sample, then take the union / rank consensus (SelectIntegrationFeatures logic; hands-on in B11)

Episode checklist

- ✓ normalize before any cross-cell comparison
- ✓ one-line check: cor(PC1, nCount) must not near 1
- ✓ pick 2,000 HVGs by vst; justify any change
- ✓ you can say what counts / data / scale.data hold
- ✓ SCT data is already logged — never log again
- ✓ switching routes = rerun downstream + record why

All six boxes ticked before you move to B7

ONE-LINE SUMMARY

Three steps solve three problems: depth, noise, magnitude.

Both routes are legitimate — pick one and walk it end to end.

If you can say what counts, data and scale.data hold and who consumes each, you have passed this episode.

Check yourself 1

What two moves does NormalizeData perform by default?

A Z-score plus clipping

B Divide by cell total, ×10,000, then log1p

C Select 2,000 highly variable genes

D Pearson residuals plus regularization

Answer B. LogNormalize = CP10K + log1p: divide depth out into proportions, then log to turn folds into sums and tame outliers. A is ScaleData, C is FindVariableFeatures, D is SCTransform.

Check yourself 2

Which downstream step mainly consumes the scale.data layer?

A FeaturePlot and VlnPlot

B FindMarkers' differential tests

C RunPCA

D saveRDS

Answer C. scale.data is the z-scored matrix prepared for PCA. Plotting and marker tests default to data; DE (B12) goes back to counts. Each layer serves its own customer.

Check yourself 3

You ran SCTransform and want a FeaturePlot. Should you log the values first?

A Yes — always log expression first

B No — SCT's data is already in log space; logging again crushes contrast

C Depends on the gene

D Yes, but with log2 instead of log1p

Answer B. SCT's data layer is log1p(corrected counts) — pre-logged at the factory. Logging again is trap 3: the positive-negative gap collapses and the plot washes out. Same for LogNormalize's data.

NEXT EPISODE

B7: PCA and choosing dimensions

Your data is in shape: twenty thousand genes about to be squeezed into a few dozen numbers.

But how many exactly? 10 or 30 — how different can it look? The answer is more forgiving than you think.